



Trailhead



[Apex Testing \(/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



[Get Started with Apex Unit Tests](#) ▾

Get Started with Apex Unit Tests

Learning Objectives

After completing this unit, you'll be able to:

- Describe the key benefits of Apex unit tests.
- Define a class with test methods.
- Execute all test methods in a class and inspect failures.
- Create and execute a suite of test classes.

Apex Unit Tests

The Apex testing framework enables you to write and execute tests for your Apex classes and triggers. Apex unit tests ensure high quality for your Apex code and let you meet requirements for deploying Apex.

Testing is the key to successful long-term development and is a critical component of the development process. The Apex testing framework makes it easy to test your Apex code. Apex code can only be written in a sandbox or a developer org, not in a production org. Apex code can be deployed to a production org from a sandbox. Also, app developers can distribute Apex code to customers from their developer orgs by uploading packages to the Salesforce AppExchange. In addition to being critical for quality assurance, Apex unit tests are also requirements for deploying and distributing Apex.

Apex unit tests lead to better code. The Apex testing framework:

- Ensures that your Apex classes and triggers work as expected.
- Has a suite of regression tests that can be rerun every time classes and triggers are updated to ensure that future updates you make to your app don't break existing functionality.
- Meets the code coverage requirements for deploying Apex to production or distributing Apex to customers via packages.





Code Coverage Requirement for Deployment

Apex Testing (/content/learn/modules/apex_testing?

Before you can deploy your code or package it for the Salesforce AppExchange, at least 75% of Apex code must be covered by tests, and all those tests must pass. In addition,

Get Started with Apex Unit Tests

each trigger must have some coverage. Even though code coverage is a requirement for deployment, don't write tests only to meet this requirement. Make sure to test the common use cases in your app, including positive and negative test cases, and bulk and single-record processing.

Test Class and Test Method Syntax

Apex test classes are annotated with the `@IsTest` 

([https://developer.salesforce.com/docs/atlas.en-](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm)

[us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_annotation_isTest.htm)). annotation. Test

classes can be defined with either private or public access modifiers

([https://developer.salesforce.com/docs/atlas.en-](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_access_modifiers.htm)

[us.apexcode.meta/apexcode/apex_classes_access_modifiers.htm](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_classes_access_modifiers.htm)). If you're using a test

class for unit testing only, declare it as private. Public test classes are typically used for

test data factory classes, which we'll cover in the Create Test Data for Apex Tests

(https://trailhead.salesforce.com/content/learn/modules/apex_testing/apex_testing_data).

unit. Classes defined as `@IsTest`  must be top-level classes.

Within the test class, test methods are also defined using the `@IsTest` 

annotation. The visibility of a test method doesn't matter, so declaring a test method as public or private doesn't make a difference, as the testing framework is always able to access test methods. For this reason, the access modifiers are omitted in the method syntax.

This code sample shows a definition of a test class with one test method.

```
1  @IsTest
2  private with sharing class MyTestClass {
3      @IsTest static void myTest() {
4          // code_block
5      }
6  }
```





Unit Test Example: Test the TemperatureConverter Class

[Apex Testing: Some Examples and Best Practices](https://trailhead.salesforce.com/content/learn/modules/apex_testing/apex_testing_intro?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

The following simple example is of a test class with three test methods. The class method that's being tested takes a temperature in Fahrenheit as an input. It converts this temperature to Celsius and returns the converted result. Let's add the custom class and its test class.

Get Started with Apex Unit Tests

1. In the Developer Console, click **File | New | Apex Class**, and enter

TemperatureConverter  for the class name, and then click **OK**.

2. Replace the default class body with the following.

```
1 public with sharing class TemperatureConverter {  
2     // Takes a Fahrenheit temperature and returns the Celsius  
3     equivalent.  
4     public static Decimal FahrenheitToCelsius(Decimal fh) {  
5         Decimal cs = (fh - 32) * 5/9;  
6         return cs.setScale(2);  
7     }  
}
```



3. Press **Ctrl+S** to save your class.

4. Repeat the previous steps to create the TemperatureConverterTest  class. Replace the default class body with the following.

```
1 @IsTest  
2 private with sharing class TemperatureConverterTest {  
3     @IsTest static void testWarmTemp() {  
4         Decimal celsius =  
5         TemperatureConverter.FahrenheitToCelsius(70);  
6         Assert.areEqual(21.11, celsius);  
7     }  
8     @IsTest static void testFreezingPoint() {  
9         Decimal celsius =  
10        TemperatureConverter.FahrenheitToCelsius(32);  
11        Assert.areEqual(0, celsius);  
12    }  
13    @IsTest static void testBoilingPoint() {  
14        Decimal celsius =  
15        TemperatureConverter.FahrenheitToCelsius(212);  
16    }
```





```
TemperatureConverter.FahrenheitToCelsius(-10);
```

The `TemperatureConverterTest` test class verifies that the method works as expected by calling it with different inputs for the temperature in Fahrenheit. Each test method verifies one type of input: a warm temperature, the freezing point temperature, the boiling point temperature, and a negative temperature. The verifications are done by calling the `Assert.areEqual()` [method](https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_class_System_Assert.htm#apex_System_Assert_areEqual_2), which takes two parameters. The first parameter is the expected value, and the second parameter is the actual value. This method can also accept a third parameter, which is a string that describes the comparison being done. This optional third parameter is used in the `testBoilingPoint()` example. The specified string is logged if the assertion fails.

Let's run the methods in this class.

1. In the Developer Console, click **Test | New Run**.
2. Under **Test Classes**, click **TemperatureConverterTest**.
3. To add all the test methods in the `TemperatureConverterTest` class to the test run, click **Add Selected**.
4. Click **Run**.
5. In the Tests tab, you see the status of your tests as they're running. Expand the test run, and expand again until you see the list of individual tests that were run. They all have green checkmarks, which indicates that all the tests passed.

Logs	Tests	Checkpoints	Query Editor	View State	Progress	Problems
Status	Test Run	Enqueued Time	Duration	Failures	Total	
✓	707o0000001cHmZ	Mon Oct 06 2014 11:53:22 GMT...		0	4	
✓	TemperatureConverterTest			0	4	
✓	testBoilingPoint		0:00			
✓	testFreezingPoint		0:00			
✓	testNegativeTemp		0:00			
✓	testWarmTemp		0:00			

After you run tests, code coverage is automatically generated for the Apex classes and triggers in the org. You can check the code coverage percentage in the Tests tab of the Developer Console. In this example, the `TemperatureConverter` class has 100% coverage, as shown in this image.



Apex Testing (/content/learn/modules/apex_testing?

trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).



Whenever you modify your Apex code, rerun your tests to refresh code

Get Started with Apex Unit Tests coverage results.

A known issue with the Developer Console prevents it from updating code coverage correctly when running a subset of tests. To update your code coverage results, use **Test | Run All** rather than **Test | New Run**.

While one test method would have resulted in full coverage of the `TemperatureConverter` class, it's still important to test for different inputs to ensure the quality of your code. Obviously, it isn't possible to verify every data point, but you can test for common data points and different ranges of input. For example, you can verify passing positive and negative numbers, boundary values, and invalid parameter values to verify negative behavior. The tests for the `TemperatureConverter` class verify common data points, such as the freezing and boiling points, as well as a positive and negative temperature.

However, the `TemperatureConverterTest` test class doesn't currently cover boundary conditions or invalid inputs. Boundary conditions are the minimum and maximum values that the method can handle. For invalid inputs, consider what happens if `null` is passed as an argument to `FahrenheitToCelsius()`. In this case, when the Apex runtime dereferences the parameter variable to evaluate the formula, it throws a `System.NullPointerException`. To handle this error, you can modify the `FahrenheitToCelsius()` method to check for an invalid input and return `null` in that case. Then add a test method to the `TemperatureConverterTest` class to verify the invalid input behavior.

Up to this point, all tests pass because the conversion formula used in the class method is correct. But that's boring! Let's try to simulate a failure just to see what happens when an assertion fails. For example, let's modify the boiling point temperature test and pass in a false expected value for the boiling point Celsius temperature (0 instead of 100). This causes the corresponding test method to fail.

1. Change the `testBoilingPoint()` test method to the following.

```
1  @IsTest
2  static void testBoilingPoint() {
3      Decimal celsius =
4      TemperatureConverter.FahrenheitToCelsius(212);
```





2. To execute the same test run, click the latest run in the Tests tab, and then click **Test** | **Rerun** from the navigation bar. The assertion in `testBoilingPoint()` fails and throws a fatal error (an `AssertException` that can't be caught).

3. Check the results in the Tests tab by expanding the latest test run. The test run report shows that out of four tests failed. To get more details about the failure, double-click the test run. Detailed results appear in a separate tab.

4. To get the error message for the test failure, double-click inside the Errors column for the failed test. The error message includes the text that we provided in the

`Assert.areEqual()` statement: `System.AssertException: Assertion Failed: Boiling point temperature is not expected.: Expected: 0, Actual: 100.00`

The test data in these test methods are numbers and not Salesforce records. You'll find out more about how to test Salesforce records and how to set up your data in the next unit.

Increase Your Code Coverage

When writing tests, try to achieve the highest code coverage possible. Don't just aim for 75% coverage, which is the lowest coverage that Salesforce requires for deployments and packages. The more test cases that your tests cover, the higher the likelihood that your code is robust. Sometimes, even after you write test methods for all your class methods, code coverage is not at 100%. One common cause is not covering all data values for conditional code execution. For example, some data values tend to be ignored when your class method has `if` statements that cause different branches to be executed based on whether the evaluated condition is met. Ensure that your test methods account for these different values.

This example includes the class method, `getTaskPriority()`, which contains two `if` statements. The main task of this method is to return a priority string value based on the given lead state. The method validates the state first and returns `null` if the state is invalid. If the state is CA, the method returns `'High'`; otherwise, it returns `'Normal'` for any other state value.

```
1 public with sharing class TaskUtil {
2     public static String getTaskPriority(String leadState) {
3         // Validate input
4         if(String.isBlank(leadState) || leadState.length() > 2) {
5             return null;
6         }
7     }
8 }
```





```
12     }  
13     return taskPriority;  
14  
15
```

[Apex Testing \(/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Get Started with Apex Unit Tests



The equality operator (`==`) performs case-insensitive string comparisons, so there is no need to convert the string to lower case first. This means that passing in `'ca'` or `'Ca'` will satisfy the equality condition with the string literal `'CA'`.

This is the test class for the `getTaskPriority()` method. The test method simply calls `getTaskPriority()` with one state (`NY`).

```
1  @IsTest  
2  private with sharing class TaskUtilTest {  
3      @IsTest  
4      static void testTaskPriority() {  
5          String pri = TaskUtil.getTaskPriority('NY');  
6          Assert.AreEqual('Normal', pri);  
7      }  
8  }
```



Let's run this test class (`TaskUtilTest`) in the Developer Console and check code coverage for the corresponding `TaskUtil` class that this test covers. After the test run finishes, the code coverage for `TaskUtil` is shown as 75%.

Open the `TaskUtil` class in the Developer Console. If you haven't already, change the setting from Code Coverage: None to **Code Coverage: All Tests**. With this setting, you'll see six blue (covered) lines and two red (uncovered) lines, as shown in this image.



Apex Testing (/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

```

6      }
7
8      String taskPriority;
9
10     if (leadState == 'CA') {
11         taskPriority = 'High';
12     } else {
13         taskPriority = 'Normal';
14     }
15
16     return taskPriority;
17 }
18 }

```

Get Started with Apex Unit Tests

The reason why line 5 isn't covered is because our test class doesn't contain a test to pass an invalid state parameter. Similarly, line 11 isn't covered because the test method doesn't pass 'CA' as the state. Let's add two more test methods to cover those scenarios. The following shows the full test class after adding the `testTaskHighPriority()` and `testTaskPriorityInvalid()` test methods. If you rerun this test class using **Test | Run All** or **Test | New Run**, the code coverage for `TaskUtil` is now at 100%!

```

1  @IsTest
2  private with sharing class TaskUtilTest {
3      @IsTest
4      static void testTaskPriority() {
5          String pri = TaskUtil.getTaskPriority('NY');
6          Assert.areEqual('Normal', pri);
7      }
8      @IsTest
9      static void testTaskHighPriority() {
10         String pri = TaskUtil.getTaskPriority('CA');
11         Assert.areEqual('High', pri);
12     }
13     @IsTest
14     static void testTaskPriorityInvalid() {
15         String pri = TaskUtil.getTaskPriority('Montana');
16         Assert.isNull(pri);
17     }
18 }

```

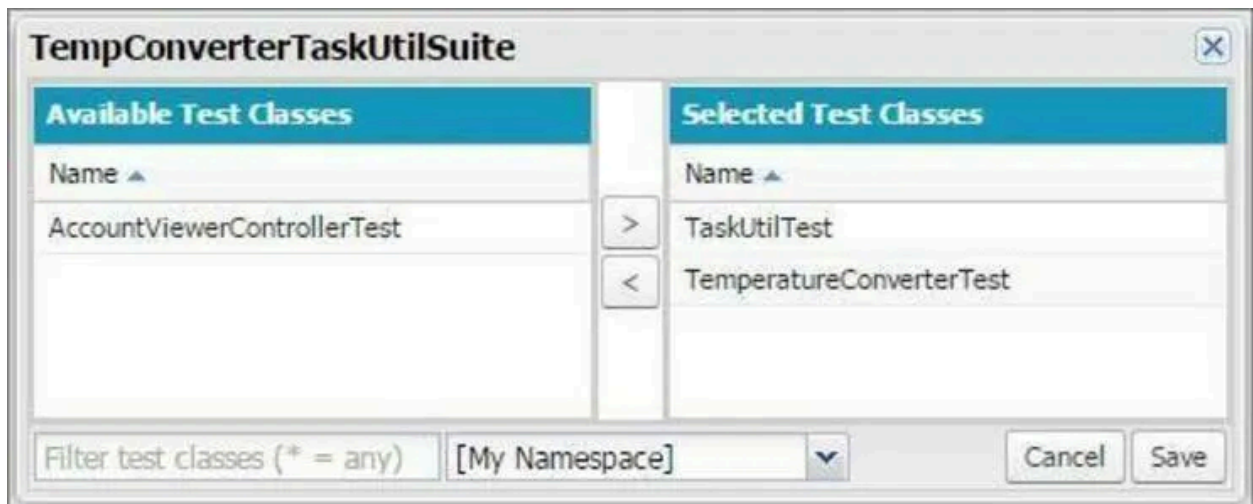




test classes that you execute together regularly.

You now have two test classes in your org. These two classes aren't related, but let's pretend for the moment that they are. Assume that there are situations when you want to run these two test classes but don't want to run all the tests in your org. Create a test suite that contains both test classes, and then execute the tests in the suite.

1. In the Developer Console, select **Test | New Suite**.
2. Enter `TempConverterTaskUtilSuite` for the suite name, and then click **OK**.
3. Select **TaskUtilTest**, hold down the Ctrl key, and then select **TemperatureConverterTest**.
4. To add the selected test classes to the suite, click **>**.



5. Click **Save**.
6. Select **Test | New Suite Run**.
7. Select **TempConverterTaskUtilSuite**, and then click **>** to move `TempConverterTaskUtilSuite` to the Selected Test Suites column.
8. Click **Run Suites**.
9. On the Tests tab, monitor the status of your tests as they're running. Expand the test run, and expand again until you see the list of individual tests that were run. Like in a run of individual test methods, you can double-click method names to see detailed test results.

Create Test Data

Salesforce records that are created in test methods aren't committed to the database. They're rolled back when the test finishes execution. This rollback behavior is handy for testing because you don't have to clean up your test data after the test executes.



Apex Testing (/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer) Even though it is not a best practice to do so, there are times when a test method needs access to pre-existing data. To access org data, annotate the test method with `@IsTest(SeeAllData=true)` [copy](#) [📄](#). The test method **Get Started with Apex Unit Tests** unit don't access org data and therefore don't use the `SeeAllData` [copy](#) [📄](#) parameter.

Tell Me More

- You can use Salesforce Apex Extension for Visual Studio Code to run Apex tests and verify the functionality of your code.
- You can save up to 6 MB of Apex code in each org. Test classes annotated with `@IsTest` [copy](#) [📄](#) don't count toward this limit.
- Even though test data rolls back, no separate database is used for testing. As a result, for some sObjects that have fields with unique constraints, inserting duplicate sObject records results in an error.
- Test methods don't send emails.
- Test methods can't make callouts to external services. You can use mock callouts in tests.
- SOSL searches performed in a test return empty results. To ensure predictable results, use `Test.setFixedSearchResults()` [copy](#) [📄](#) to define the records to be returned by the search.

Resources

- [Apex Developer Guide: Testing Best Practices](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_best_practices.htm)
(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_best_practices.htm)
- [Apex Developer Guide: What Are Apex Unit Tests?](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_unit_tests.htm)
(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_unit_tests.htm)
- [Apex Developer Guide: Isolation of Test Data from Organization Data in Unit Tests](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_data_access.htm)
(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_data_access.htm)
- [Salesforce Help: Checking Code Coverage](https://help.salesforce.com/s/articleView?id=platform.code_dev_console_tests_coverage.htm&language=en_US)
(https://help.salesforce.com/s/articleView?id=platform.code_dev_console_tests_coverage.htm&language=en_US)



```
1 public with sharing class VerifyDate {
2     //method to handle potential checks against two dates
3     public Date SetEndOfMonthDate(Date date1, Date date2) {
4         //if date2 is within 30 days of date1, use date2.
5         Otherwise use the end of the month
6         if(DateWithin30Days(date1,date2)) {
7             return date2;
8         } else {
9             return SetEndOfMonthDate(date1);
10        }
11    }
12    //method to check if date2 is within the next 30 days of date1
13    private static Boolean DateWithin30Days(Date date1, Date
14    date2) {
15        //check for date2 being in the past
16        if( date2 < date1) { return false; }
17        //check that date2 is within (>=) 30 days of date1
18        Date date30Days = date1.addDays(30); //create a date 30 days
19        away from date1
20        if( date2 >= date30Days ) { return false; }
```



Apex Testing (/content/learn/modules/apex_testing?

trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Get Started with Apex Unit Tests

Show More

GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

YOUR CHALLENGE

Create a Unit Test for a Simple Apex Class

Create a simple Apex class to test if a date is within a proper range, and if not, returns a date that occurs at the end of the month within the range. You'll copy the code for the class from the Get Ready for the Hands-on



- Code: **Copy from Get Ready for the Hands-on Challenge section above**

Apex Testing (/content/learn/modules/apex_testing?

trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

- Name: [copy](#)

Get Started with Apex Unit Tests

- Goal: 100% code coverage
- Run your test class at least once

Choose hands-on org

Koushik nelluri Salesforce CRM

Created on 5/9/2026



Launch

Check Challenge to Earn 500 Points

Learn

Badges

Trails

Trailmixes

Superbadges

Trailhead Academy

Career Paths

Certifications

Certifications

Maintain Certifications

Verify Certifications

Community

Trailblazer Community

Events

Agentblazers

Salesforce Developers

Salesforce Admins

Code of Conduct

Report Illegal Content
(EU)

Extras

Trailhead Login

Sales Enablement

Trailblazer Store

Salesforce Help

Download Trailhead GO





[Apex Testing \(/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Get Started with Apex Unit Tests